

整合練習 4：Data Conversion 觀念總覽

Lecture

if01 ~ if03 都在講「呼叫端跟 SAP 系統之間，一次一筆的正常業務呼叫」。這題換一個情境：**資料轉換 / 搬遷 (Data Conversion)**——舊系統上線初期，要把舊系統累積多年的主檔 / 交易資料 (客戶、供應商、物料、庫存、歷史訂單.....) **一次性大量灌進新系統**，或是系統上線後跟其他系統做**批次資料同步**。這種情境的資料量動輒幾萬到幾百萬筆，「怎麼把資料寫進去」本身就是一個要慎重選型的技術決策，不是隨便找個 **INSERT** 語句就打發。

先講清楚一個容易誤解的前提：SAP 的業務資料 (客戶主檔、物料主檔.....) 背後往往牽動一大串**業務規則檢查**——欄位之間的關聯驗證、Customizing 決定的必填欄位、衍生欄位的自動計算、權限檢查.....這些邏輯絕大多數寫在**畫面處理邏輯 (Dynpro Module Pool) 或 BAPI / Function Module 內部**，不是寫在資料庫的 Check 或 Trigger 裡。基礎課 ex21 已經教過一個具體案例：DDIC 外鍵 (**WITH FOREIGN KEY**) 只在畫面輸入 (SM30/Dynpro) 層級生效，直接下 Open SQL **INSERT** 完全不受外鍵約束——**這個現象不是 bug，是 SAP 資料驗證架構的常態**：業務邏輯活在應用層，不是資料庫層。這代表如果資料轉換程式選擇「直接組資料 **INSERT / UPDATE** 進標準表」，等於**繞過了所有這些應用層驗證**，資料轉進去了，但完全沒人保證它符合業務規則——這是實務上最常踩的坑，也是這題要建立的第一個觀念防線。

四種主流資料轉換技術，依「有沒有繞過業務規則檢查」跟「速度」兩個軸排開：

1. **手動 Open SQL (INSERT/UPDATE/MODIFY)** ——**不建議**：最快、最直覺，但完全繞過業務規則檢查 (就是上面 ex21 那個外鍵案例的直接後果)，資料一致性全靠轉換程式自己重新實作一遍驗證邏輯 (等於把 SAP 標準交易的業務邏輯抄一份、還要維護)，維護成本高、風險大，只適合真正單純、沒有複雜業務規則的自訂 Z 表。
2. **BDC (Batch Data Communication，也叫 Batch Input)** ——**舊但穩**：用 **BDCDATA** 內部表模擬使用者在畫面上的按鍵與輸入 (**SHDB** 錄製產生骨架)，實際執行時真的把資料「打」進標準交易的畫面流程，所以畫面層的所有驗證邏輯都會照跑一遍——這是它比 Open SQL 可靠的根本原因 (跟 ex21 外鍵案例正好相反：BDC 模擬畫面輸入，外鍵檢查會生效)。缺點是慢 (要逐筆模擬整個畫面流程)、脆弱 (畫面版面改了、加了自訂欄位，BDC 腳本可能就要跟著改)。if05 會實際動手寫一支。
3. **Direct Input**——**SAP 官方提供、限定特定物件的高速轉檔程式**：SAP 針對少數幾個「資料轉換需求特別大」的標準物件 (客戶主檔、供應商主檔、物料主檔、總帳科目.....)，另外寫了一套**繞過畫面、直接呼叫底層驗證與寫入邏輯**的專用程式，速度比 BDC 快得多，同時業務規則檢查沒有少 (因為呼叫的還是同一套底層邏輯，只是跳過了畫面渲染這一層)。這次在系統上實際查證到的例子：

| Direct Input 程式 | 對應物件 | 套件 | |---|---|---| | **RMDATIND** | 物料主檔 | **MGAD** | | **RFBIDE00** | 客戶主檔 | **VS** | | **RFBIKR00** | 供應商主檔 | **WLIF** |

限制很明確：只有 SAP 覺得值得投資做這件事的物件才有對應的 Direct Input 程式，你自訂的 Z 物件、或 SAP 沒特別做過的標準物件，沒有這個選項。

1. **BAPI-based conversion**——**目前主流**：直接呼叫該物件對應的 BAPI (如果有的話)。跟 Direct Input 一樣不經過畫面 (快)，跟 BDC 一樣會跑完整業務規則檢查 (因為 BAPI 內部本來就是在做這件事)，而且介面比 Direct Input 更穩定、更有文件 (if01 教過的 Release 承諾)。這正是本課程 if02 / if03 已經示範過的路數 (**BAPI_INSPLT_SETUSAGEDECISION**、**BAPI_GOODSMVT_CREATE**)，也是 REST 課 rs10 批次呼叫 **BAPI_FLBOOKING_CREATEFROMDATA** 的設計理由。

四種技術總表：

技術	速度	業務規則檢查	適用範圍	介面穩定度
Open SQL 直寫	最快	無（完全繞過）	任何表（但不建議用在有業務邏輯的物件）	不穩定（自己維護，SAP 升級可能整套邏輯要重寫）
BDC（Batch Input）	慢（逐筆模擬畫面）	有（畫面層全跑一遍）	任何有畫面的交易，範圍最廣	中等（畫面改版 / 加欄位可能斷）
Direct Input	快	有（底層邏輯照跑，只跳過畫面）	侷限（只有 SAP 做好的特定物件才有）	較穩定（SAP 官方程式，但通常沒有 BAPI 等級的公開文件）
BAPI-based	快	有（BAPI 本身就是在做業務規則檢查）	侷限（只有掛 BAPI 的物件才有）	高（if01 教過的 Release 承諾）

LSMW（Legacy System Migration Workbench）不是第五種技術，是一個包裝工具：LSMW 是一個 GUI 精靈（交易碼 **LSMW**），讓使用者用畫面設定「來源欄位怎麼對應到目標欄位」，但它底層執行資料寫入時，用的還是上面四種技術之一（Batch Input、Direct Input、BAPI、或 IDoc）——LSMW 的價值在於「把欄位對應、資料清洗、執行監控這些繁瑣工作包成一個標準化流程」，不是發明了新的寫入機制。LSMW 本身是傳統 Dynpro-based 工具，這次沒有查到對應的 ADT API（合理，跟課程已知的 SM59 / DBCO 屬於同一種 GUI-only 限制）。**補充一個現況：**較新的 S/4HANA 系統上，LSMW 逐漸被 **Migration Cockpit**（交易碼 **LTMC/LTMOM**）取代，Migration Cockpit 預設就是走 BAPI / API 為基礎的轉換方式——這代表 SAP 自己的工具演進方向，也是朝著「四種技術裡優先用 BAPI」這個原則走，這門課範圍不深入 Migration Cockpit，但值得知道這個趨勢。

學習目標

- 能講出四種資料轉換技術（Open SQL / BDC / Direct Input / BAPI-based）的取舍：速度、是否繞過業務規則檢查、適用範圍、介面穩定度
- 能解釋「為什麼 Open SQL 直寫在正式資料轉換情境不建議」，並用 ex21 學過的 DDIC 外鍵案例佐證
- 知道 Direct Input 程式的角色與限制（只有 SAP 覺得值得投資的特定物件才有）
- 理解 LSMW 是包裝工具、不是第五種技術，底層仍然是上述四者之一
- 給定一個實務情境，能判斷該選哪種技術並說明理由

事前準備

不需要新建任何 SAP 物件，這題是觀念比較與情境判斷。三支 Direct Input 範例程式已於本次連線（client 130）用 **sap-adt** MCP 的 quickSearch 查證確實存在於這套系統：**RMDATIND**（套件 **MGAD**）、**RFBIDE00**（套件 **VS**）、**RFBIKR00**（套件 **WLIF**），有興趣可以在 SE38 打開看看程式規模（都是幾千行等級的大型程式，不要求精讀，看個規模感即可）。

題目需求

- 完成 **Lecture** 的四技術比較表（可以直接照抄 Lecture 的表格，重點是理解每一格背後的理由，不是死背）。

2. 情境判斷 (針對下面三個情境，選一種技術並說明理由)：

- 情境一：舊系統要匯入 10 萬筆客戶主檔到新系統，一次性轉換，之後不會再用同一支程式
- 情境二：有一個完全自訂的 Z 表 (沒有畫面、沒有 BAPI、也沒有 Direct Input)，要匯入一批歷史資料
- 情境三：正式上線後，要跟另一套系統即時同步訂單資料 (不是一次性搬遷，是長期、高頻率的資料交換)

3. **LSMW 定位題**：說明「LSMW 執行資料寫入時選 Batch Input 方式」跟「LSMW 執行資料寫入時選 BAPI 方式」，兩者在速度與業務規則檢查上的差異，應該跟 Lecture 表格裡哪兩列一致？

參考答案 (情境判斷)

- **情境一**：優先看有沒有現成 Direct Input 程式 (客戶主檔剛好有 **RFBIDE00**)，有的話直接用 (或包在 LSMW 專案裡跑)；一次性、大量、有官方程式支援的情境是 Direct Input 的典型場景。
- **情境二**：沒有 BAPI、沒有 Direct Input，如果這個 Z 表有對應的維護畫面 (例如 SM30 Table Maintenance)，可以考慮 BDC 模擬畫面輸入；如果連畫面都沒有、資料驗證邏輯很單純，也可以評估直接寫一支自訂的驗證程式再用 Open SQL 寫入 (但要自己完整重新實作驗證邏輯，這正是 Lecture 提到的「不建議」但「有時候是唯一選項」的情境)。
- **情境三**：長期、高頻率、需要即時性——BAPI-based 是最適合的選項，介面穩定、速度快、業務規則檢查沒有少；這也是本課程 if02 / if03 跟 REST 課 rs10 一路示範的路數。

團隊實務備註

- **RMDATIND / RFBIDE00 / RFBIKR00** 這三支都已用 **sap-adt** MCP 的 quickSearch 在 client 130 查證存在，這代表「找 Direct Input 程式」也可以用 if02 教過的方法論 (先猜命名規則、或搜關鍵字) 去找，不是只能翻 SAP 文件；程式名稱慣例上常見 **R + 模組代碼 + BI + 描述** 這種老式命名 (Direct Input 技術本身是 R/2 時代就有的老技術，比 BAPI 更早出現，命名風格也帶著那個年代的痕跡)。
- LSMW 本身這次沒有查到對應的 ADT API，合理判斷屬於本專案已知的 GUI-only 類別 (跟 SM59 / DBCO / SWO1 同一種限制)，出題時若要示範 LSMW 操作，只能靠使用者在 SAP GUI 手動操作，Claude 端無法自動化建立或驗證 LSMW 專案。
- 這題沒有查證 Migration Cockpit (**LTMC/LTMOM**) 在這套系統是否已啟用 / 有沒有 ADT API，只是順帶一提現況趨勢，不列入本題的驗收範圍。

思考題

1. 為什麼 SAP 不乾脆讓 Direct Input 支援所有物件，省得還要有 BDC 這種相對笨重的技術？ (提示：Direct Input 要繞過畫面、直接呼叫底層邏輯，代表 SAP 得為每一個物件重新設計一套「跳過畫面但驗證效果等價」的邏輯——這個開發成本很高，只有轉換需求量大到值得投資的物件，SAP 才會做；冷門物件沒有專門的 Direct Input，只能退回 BDC 模擬畫面)
2. 如果一個物件同時有 Direct Input 程式又有 BAPI，該選哪個？ (提示：通常優先選 BAPI——介面更現代、有 if01 教過的 Release 承諾、通常也支援單筆即時呼叫而不只是批次轉檔；Direct Input 是為了「一次性大量轉檔」設計的老技術，新的整合情境現在多半走 BAPI)
3. ex21 教過 DDIC 外鍵只在畫面輸入層級生效、Open SQL 不受影響——這跟「為什麼 BDC 比 Open SQL 可靠」是不是同一個道理？ (提示：是的，BDC 正是靠「模擬畫面輸入」這件事，把包含外鍵檢查在內的所有畫面層驗證邏輯都跑一遍，這是它比 Open SQL 慢、但資料品質更有保障的根本原因)

答案

不新建任何 SAP 物件；三支 Direct Input 範例程式 (**RMDATIND** / **RFBIDE00** / **RFBIKR00**) 已於本次連線系統 (client 130) 查證存在，比較表與情境判斷見本題內文。